Global Warming Science

https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/

Ocean circulation collapse

Please use the template below to answer the workshop questions. "XX" indicates places where you need to complete/write code or add a discussion.

your name:

[ ]:
```
# import needed libraries:
# ------------------------
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
from scipy.integrate import solve_ivp
import pickle
import datetime

# load data from pickle file:
# ---------------------------
with open('./ocean_circulation_variables.pickle', 'rb') as file:
    d = pickle.load(file)
    # print information about each extracted variable:
    for key in list(d.keys()):
        print(f"extracting pickled variable: name={key}; size={d[key].shape}")
        #print("; type=",type(d[key]))
    globals().update(d)
```

Explanation of variables:

RAPID time series and time axis (given both as dates and as time in days):

RAPID_AMOC_data, RAPID_time, RAPID_dates

RAPID monthly climatology and parameters of a fit to a sine function:

RAPID_AMOC_monthly_climatology, RAPID_sine_fit_coeffs,

The sine fit is given as $a_0 + a_1 \sin(2\pi(t_{years} + a_2)/365)$, where a=RAPID_sine_fit_coeffs and t_{years} is RAPID_time/365.

AMOC stream function for 1st and last decades of 21ast century as a function of depth and latitude, with corresponding axes:

rcp85_AMOC_first_decade, rcp85_AMOC_last_decade, rcp85_depth, rcp85_latitude

Time series of AMOC projections and corresponding time axis:

rcp85_AMOC_timeseries, rcp85_years

AMOC value from ship obs and the corresponding times:

ship_obs_AMOC, ship_obs_months, ship_obs_years

1) Observations: Has the AMOC decreased already over the past decades?

1a) Estimate the heat transport due to AMOC (Watts) by assuming it to be composed of poleward transport of 20 Sv in the upper 1000 m and a return flow below. Assume each of

these two flows carries water of a temperature that represents the average over these two depth ranges in the North Atlantic, as seen in the figure in Box 6.1. Compare to the world energy consumption per second.

[ ]:
```
# heat transport
c_p=XX # J/(kg C)
# mass transport is 20 Sv or XX kg/s:
# heat transport = mass transport (kg/s) * c_p (J/kg) * (T_{upper ocean} -T_{lower␣
↪ocean}) (K)
heat_transport=XX

# The annual global energy consumption (J/s):
world_consumption=XX

print(
    f" AMOC transport={heat_transport:g}"
    f"\n world concumption per second={world_consumption:g}"
    f"\n AMOC/world consumption={heat_transport/world_consumption:g}"
)
```

1b) Draw the AMOC estimate based on historical ship observations as a function of year.

[ ]:
```
# plot ship_obs data:
# ------------------
plt.figure(figsize=(4,3),dpi=300)
plt.plot(XX,XX,'-x',markersize=10)
plt.xlabel("time (years)")
plt.ylabel("Sv");
plt.title("ship obs");
```

1c) Draw the observed RAPID AMOC time series and the seasonal sine fit to it.

[ ]:
```
# plot AMOC obs:
fig=plt.figure(figsize=(3.8,1.8),dpi=300)
plt.plot(XX,XX,color="b",label="RAPID obs, smoothed")
plt.xlabel("Year")
plt.ylabel("AMOC (Sv)")

# add sine fit: sine_fit=a_0+a_1*sin(2*pi*(time+a_2)/365)
a=1.0*RAPID_sine_fit_coeffs
RAPID_AMOC_sine_fit=XX
plt.plot(XX,XX,color="g",label="sine fit")
plt.legend()
plt.title("RAPID AMOC time series and sine fit to the data")
plt.show();
```

1d) Plot the monthly climatology of RAPID and the sine fit. Superimpose the few available AMOC transport estimates based on historical ship observations as if all observations were taken during different months of one year. What is your conclusion regarding the weakening of AMOC deduced from ship observations?

[ ]:
```
# Plot one year of AMOC monthly climatology with ship_obs data superimposed:
# ---------------------------------------------------------------------------
fig=plt.figure(figsize=(4,3),dpi=300)
time_months=(0.5+np.arange(0,12))
time_sine_fit_months=np.arange(0,12,0.1)
```

```
plt.plot(time_months,XX,'x',color=XX,markersize=15,label="RAPID monthly climatology")
sine_fit_one_year=XX
plt.plot(time_sine_fit_months,sine_fit_one_year,label="sine fit".color=XX)

# superimpose ship observations:
X=ship_obs_months[:]
Y=ship_obs_AMOC[:]
plt.scatter(XX,XX,label="AMOC ship obs")
for i in range(len(X)):
    plt.text(X[i],Y[i],str(ship_obs_years[i]))
plt.xlabel("Time (months)")
plt.ylabel("Sverdrup ($10^6$ ton/s)")
plt.xlim(0,12)
plt.legend();
plt.show()
```

Discuss: XX

2) Projections:

2a) Plot the RCP8.5 projected AMOC time series (of maximum stream function) until 2100.

[ ]:
```
plt.figure(figsize=(8,4),dpi=200)
plt.plot(XX,XX)
plt.xlabel("year")
plt.ylabel("Sv")
plt.title("max AMOC time series")
plt.show()
```

2b) Contour the AMOC streamfunction averaged over the first decade of the RCP8.5 scenario (2006–2016), as well as averaged over the last decade of this century (2090–2100). Describe the differences.

[ ]:
```
plt.figure(figsize=(12,6))
levels=np.arange(-8,28,2) # in Sverdrup, adjust as needed

plt.subplot(1,2,1)
plt.contourf(rcp85_latitude,rcp85_depth,rcp85_AMOC_first_decade,levels)
plt.gca().invert_yaxis()
plt.colorbar()
plt.title("AMOC first decade")
plt.xlim(-40,90)

plt.subplot(1,2,2)
XX contourf last decade and add colorbar as in the first subplot above
plt.title("AMOC, last decade")
plt.xlim(-40,90)

plt.tight_layout()
plt.show();
```

discussion: XX

3) Stommel model:

3a) Plot the steady states of the AMOC for freshwater forcings of $F_s = 0, \ldots, 5$ m/year. Describe the number and stability of the solutions as a function of F_s.

[ ]:
```
# first cell out of three: setup parameters and needed functions:
# ----------------------------------------------------------------

# set parameters:
meter=1;
year=365*24*3600;
S0=35.0;
DeltaT=-10;
time_max=10000*year;
alpha=0.2; # (kg/m^3) K^-1
beta=0.8; # (kg/m^3) ppt^-1
k=10e9 # (m^3/s)/(kg/m^3) #  3e-8; # sec^-1
# area of each of the ocean boxes, rediculous value, needed to get
# all other results at the right order of magnitude:
area=(50000.0e3)**2
depth=4000
V=area*depth # volume of each of the ocean boxes
Sv=1.e9 # m^3/sec


#############################################################################
def q(DeltaT,DeltaS):
    #########################################################################
    # THC transport in m^3/sec as function of temperature and salinity
    # difference between the boxes
    flow=XX # this is "q" from the course notes
    return flow

#############################################################################
def steady_states(Fs,X):
    #########################################################################
    # 3 steady solutions for Y:
    y1=XX
    if XX>0:
        y2=XX
        y3=XX
    else:
        y2=np.nan
        y3=np.nan
    Y=np.array([y1,y2,y3])
    return Y

#############################################################################
def Fs_func(time,time_max,is_Fs_time_dependent):
    #########################################################################
    # total surface salt flux into northern box

    #########################################################################
    # Specify maximum and minimum of freshwater forcing range during the
    # run in m/year:
```

```
    FW_min=-0.1;
    FW_max=5;
    ##########################################################################
    if is_Fs_time_dependent:
            flux=FW_min+(FW_max-FW_min)*time/time_max;
    else:
        flux=2;

    # convert to total salt flux:
    return flux*area*S0/year

print("done procedssing function definitions.")
```

[ ]:
```
# second cell out of three: calculate:
# -------------------------------------

# find steady states, including unstable ones, as function of F_s:
# ----------------------------------------------------------------
Fs_range=np.arange(0,5,0.01)*area*S0/year
# set up two arrays for results, to be calculated in loop below:
DeltaS_steady=np.zeros((3,len(Fs_range)))
q_steady=np.zeros((3,len(Fs_range)))

i=0
for Fs in Fs_range:
    Y=steady_states(Fs,alpha*DeltaT)
    # translate Y solution to a solution for DeltaS:
    DeltaS_steady[:,i]=XX
    for j in range(0,3):
        q_steady[j,i]=q(DeltaT,DeltaS_steady[j,i])
    i=i+1

print(
    f"q_steady.shape={q_steady.shape}; "
    f"Example q_steady values={q_steady[:,100]}"
)
```

[ ]:
```
# third cell out of three: plot
# -----------------------------

plt.figure(1,figsize=(8,4),dpi=200)

Fs_to_m_per_year=S0*area/year
plt.subplot(1,2,1)
# plot all three solutions for Delta S as function of Fs in units of m/year:
plt.plot(XX,XX,'r.',markersize=1)
plt.plot(XX,XX,'g.',markersize=1)
plt.plot(XX,XX,'b.',markersize=1)
# plot a dash think line marking the zero value:
plt.plot(XX,XX,'k--',dashes=(10, 5),linewidth=0.5)
plt.title('(a) steady states')
plt.xlabel('$F_s$ (m/year)');
plt.ylabel(r'$\Delta S$');
```

```
plt.xlim([min(Fs_range/Fs_to_m_per_year), max(Fs_range/Fs_to_m_per_year)])

plt.subplot(1,2,2)
# plot all three solutions for q (in Sv) as function of Fs in units of m/year:
plt.plot(XX,XX,'r.',markersize=1,label="lower stable branch")
plt.plot(XX,XX,'g.',markersize=1,label="upper stable branch")
plt.plot(XX,XX,'b.',markersize=1,label="unstable branch")
plt.plot(XX,XX,'k--',dashes=(10, 5),linewidth=0.5)
plt.title('(b) steady states')
plt.xlabel('$F_s$ (m/year)');
plt.ylabel('$q$ (Sv)');

plt.tight_layout()
plt.show()
```

3b) Integrate the time dependent differential equation for S in a scenario of slowly increasing fresh water forcing, plot the resulting AMOC (q) as function of time.

[ ]:
```
###########################################################################
def rhs_S(time,DeltaS,time_max,DeltaT,is_Fs_time_dependent):
    ###########################################################################
    # right hand side of the equation for d/dt(DeltaS):
    # Input: q in m^3/sec; FW is total salt flux into box
    rhs=XX;
    return rhs/V


# next, do a time dependent run:
# ------------------------------
is_Fs_time_dependent=True;
y0=[0]
teval=np.arange(0,time_max,time_max/1000)
tspan=(teval[0],teval[-1])
sol = solve_ivp(fun=lambda time,DeltaS:␣
 ↪rhs_S(time,DeltaS,time_max,DeltaT,is_Fs_time_dependent) \
                ,vectorized=False,y0=y0,t_span=tspan,t_eval=teval)
Time=sol.t
DeltaS=sol.y

is_Fs_time_dependent=True;
FWplot=np.zeros(len(Time))
qplot=np.zeros(len(Time))
i=0;
for t in Time:
    FWplot[i]=Fs_func(t,time_max,is_Fs_time_dependent);
    qplot[i]=q(DeltaT,DeltaS[0,i]);
    i=i+1;

N=len(qplot);


###########################################################################
## plots:
###########################################################################
```

```
plt.figure(figsize=(12,4),dpi=200)
plt.subplot(1,3,1)
plt.plot(Time/year/1000,FWplot/Fs_to_m_per_year,'b-',markersize=1)
plt.plot(Time/year/1000,0*Time,'k--',dashes=(10, 5),linewidth=0.5)
plt.xlabel('time (kyr)');
plt.ylabel('$F_s$ (m/yr)');
plt.title('(d) $F_s$ for time-dependent run');

plt.subplot(1,3,2)
plt.plot(Time/year/1000,qplot/Sv,'b-',markersize=1)
plt.plot(Time/year/1000,Time*0,'k--',dashes=(10, 5),lw=0.5)
plt.xlabel('time (kyr)');
plt.ylabel('AMOC $q$ (Sv)');
plt.title('(e) AMOC transport $q$');

plt.subplot(1,3,3)
plt.plot(Time/year/1000,DeltaS[0,:],'b-',markersize=1)
plt.title(r'(f) $\Delta S$ vs time');
plt.xlabel('Time (kyr)');
plt.ylabel(r'$\Delta S$');


plt.tight_layout()
plt.show()
```

discussion: XX

3c) Plot d S/dt as function of S and discuss the stability of the three possible solutions.

[ ]:
```
# calculate dDeltaS/dt as function of DeltaS for stability analysis:
# ------------------------------------------------------------------

# use fixed (time-independent) value of fresh water forcing for this case:
is_Fs_time_dependent=False
time=0
DeltaS_range=np.arange(-3,0,0.01)
rhs=np.zeros(len(DeltaS_range))
for i in range(0,len(DeltaS_range)):
    DeltaS = DeltaS_range[i]
    rhs[i]=rhs_S(time,DeltaS,time_max,DeltaT,is_Fs_time_dependent)
    i=i+1
rhs=rhs/np.std(rhs)


plt.figure(figsize=(5,4),dpi=200)
plt.plot(DeltaS_range,rhs,'k-',lw=2)
plt.plot(DeltaS_range,rhs*0,'k--',dashes=(10, 5),lw=0.5)
# superimpose color markers of the 3 solutions
Fs=Fs_func(0.0,0.0,False)
yy=steady_states(Fs,alpha*DeltaT)/beta
plt.plot(yy[0],0,'ro',markersize=10)
plt.plot(yy[1],0,'go',markersize=10)
plt.plot(yy[2],0,'bo',markersize=10,fillstyle='none')
```

```
plt.title('(c) stability')
plt.xlabel(r'$\Delta S$');
plt.ylabel(r'$d\Delta S/dt$');
```

discussion: XX

3d) Explain how tipping points and hysteresis may occur in this model.

XX explanation